

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения „Книга снов“ с личными кабинетами пользователей для ведения дневника сновидений и статистического анализа»

Выполнил: Кленечева Анастасия Владимировна

Группа: 241-327

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

УТВЕРЖДАЮ

заведующая кафедрой

«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«_____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Кленечевой Анастасии Владимировне,

обучающейся группы 241-327,

направления подготовки 09.03.01 «Информатика и вычислительная техника»
по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения „Книга снов“ с личными кабинетами
пользователей для ведения дневника сновидений и статистического анализа».

1. Исходные данные к работе (проекту): информационные ресурсы в сети
интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов,
подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих решений	10	22.03.2026	Анализ аналогов
Задача 1.2. Анализ инструментов разработки	7	26.03.2026	Выбор стека
Задача 1.3. Формулировка цели и задач	8	30.03.2026	Постановка ТЗ
Раздел 2. Проектирование			
Задача 2.1. Анализ целевой аудитории	5	02.04.2026	Портрет пользователя
Задача 2.2. Описание функциональности	7	06.04.2026	
Задача 2.3. Проектирование модели данных	8	10.04.2026	ER-диаграмма

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.4. Разработка макетов страниц	5	14.04.2026	
Раздел 3. Разработка			
Задача 3.1. Модели БД	5	17.04.2026	
Задача 3.2. Аутентификация и личные кабинеты	5	21.04.2026	
Задача 3.3. CRUD для снов	6	25.04.2026	Базовый функционал
Задача 3.4. Загрузка файлов	6	29.04.2026	
Задача 3.5. Анализ текста сна	6	04.05.2026	Поиск символов
Задача 3.6. Агрегация данных и статистика	7	09.05.2026	Аналитика
Задача 3.7. Экспорт в CSV	4	19.05.2026	Отчёт
Задача 3.8. Панель администратора	6	24.05.2026	Модерация
Раздел 4. Оформление итогов			
Задача 4.1. Git-репозиторий	3	22.03.2026	
Задача 4.2. Деплой на хостинг	4	26.05.2026	Публикация
Задача 4.3. Оформление отчёта	3	26.05.2026	Пояснительная записка

Руководитель курсовой работы (проекта):

старший преподаватель кафедры «Инфокогнитивные технологии»

«____» _____ 2026 г.

(подпись) А. С. Кружалов

Дата выдачи задания

«30» марта 2026 г.

Дата сдачи выполненной работы

«13» июня 2026 г.

Задание приняла к исполнению

«30» марта 2026 г.



(подпись)

А. В. Кленечева

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	8
1.1 Обзор существующих решений	8
1.2 Анализ инструментов разработки	9
1.3 Формулировка цели и задач	11
2 ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	12
2.1 Анализ целевой аудитории	12
2.2 Описание функциональности приложения	13
2.3 Проектирование модели данных	15
2.4 Разработка макетов страниц	17
ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ	23
3.1 Модели базы данных	23
3.2 Аутентификация и личные кабинеты	24
3.3 CRUD-интерфейс для управления снами	27
3.4 Загрузка файлов	30
3.5 Анализ текста сна	32
3.6 Агрегация данных и статистика	33
3.7 Экспорт в CSV	34
3.8 Панель администратора	35
4 ОФОРМЛЕНИЕ ИТОГОВ РАБОТЫ	37
4.1 Git-репозиторий	37
4.2 Деплой на хостинг	37
4.3 Оформление отчёта	38
ЗАКЛЮЧЕНИЕ	39
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	41

ВВЕДЕНИЕ

Проблема исследования. Сновидения являются важным источником информации о психоэмоциональном состоянии человека и его подсознательных процессах. Ведение дневника сновидений традиционно используется в психотерапии и самоанализе. Однако существующие способы фиксации снов — бумажные дневники, текстовые файлы или приложения-заметки — не позволяют автоматически анализировать содержание снов, выявлять повторяющиеся символы или визуализировать статистику. Таким образом, проблема заключается в отсутствии доступного специализированного инструмента, сочетающего личный дневник сновидений с аналитикой данных.

Актуальность темы. В последние годы наблюдается рост интереса к велнес-технологиям, направленным на поддержание психического здоровья и саморазвитие. Возможность вести структурированный дневник снов и получать статистический анализ своих записей повышает вовлечённость пользователя и способствует более глубокому осмыслению сновидений. Разработка веб-приложения, предоставляющего каждому пользователю личную «Книгу снов» с возможностью анализа и визуализации данных, является актуальной задачей на стыке психологии, велнес-технологий и веб-разработки.

Степень разработанности проблемы. В настоящее время существуют отдельные решения для ведения дневников сновидений: мобильные приложения «Elsewhere», «Dreamov», «Awoken». Анализ аналогов показывает, что большинство из них либо перегружены платными функциями, либо примитивны и не предоставляют возможностей для анализа данных. Приложения, поддерживающие статистический анализ, встречаются редко. Таким образом, создание веб-приложения с концепцией «Книги снов», объединяющей дневник и аналитику, является практической задачей.

Объект исследования — процессы ведения, систематизации и анализа дневников сновидений с использованием веб-технологий и генеративных нейросетей.

Предмет исследования — программные средства для создания веб-приложения «Книга снов», обеспечивающего личные кабинеты пользователей, CRUD-операции с записями, статистическую обработку текстов снов и экспорт данных.

Целью данной работы является проектирование и реализация клиент-

серверного веб-приложения «Книга снов» для ведения дневника сновидений с личными кабинетами и статистическим анализом.

Задачи исследования:

- провести анализ существующих решений и обосновать выбор технологий;
- спроектировать архитектуру приложения и модель данных;
- разработать систему аутентификации и разграничения прав доступа по ролям;
- реализовать CRUD-интерфейс для управления записями снов;
- разработать модуль агрегации данных для статистического анализа;
- реализовать экспорт данных в CSV;
- выполнить оформление итогов работы.

Практическая значимость работы. Разработанное веб-приложение может быть использовано для ведения личного дневника сновидений, а также психологами для сбора и анализа данных. Анализ символов и эмоционального фона позволяет пользователю лучше понимать свои подсознательные процессы.

АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих решений

Для определения места разрабатываемого приложения на рынке и выявления недостатков аналогов проведём анализ существующих решений для ведения дневников сновидений. На сегодняшний день наиболее популярными являются мобильные приложения, так как они позволяют делать запись сразу после пробуждения. Рассмотрим три характерных представителя.

Dreamovy (Android/iPhone). Приложение с акцентом на категоризацию снов по темам (страхи, мечты, повторяющиеся сны). Позволяет вести дневник, ставить оценки эмоциональному фону, имеет базовую статистику (календарь снов). Бесплатная версия ограничена количеством записей, реклама. Плюсы: удобная категоризация. Минусы: нет анализа символов, платный доступ к расширенной статистике.

Awoken (Android). Приложение для практикующих осознанные сновидения. Содержит напоминания для выполнения «проверок реальности», звуковые сигналы, а также дневник с тегами. Полностью бесплатное. Плюсы: удобно для пользователей, открытый исходный код. Минусы: очень простая аналитика — только текстовый поиск по дневнику, нет графиков, нет визуализации символов.

Lucid Dreamer (iPhone). Платное приложение с акцентом на аудио-дневник (запись голосом сразу после пробуждения). Поддерживает облачное резервное копирование. Плюсы: удобный голосовой ввод, красивое оформление. Минусы: отсутствие статистического анализа, нет экспорта данных, подписка стоит около 3000 руб./год.

На основе анализа выделим ключевые критерии, важные для пользователей: поддержка аналитики символов, визуализация данных, наличие тегов/категорий, экспорт данных, стоимость, веб-доступ. Сравнение представлено в таблице 1.1.

Анализ таблицы показывает, что ни одно из рассмотренных приложений не предоставляет полноценного статистического анализа текста снов (частотность символов, эмоциональные тренды), не поддерживает экспорт данных в PDF и не имеет веб-интерфейса. Приложение «Книга снов» призвано заполнить эти пробелы, сочетая личный дневник с аналитикой и доступом через браузер.

Таблица 1.1 – Сравнительный анализ приложений для ведения дневников сновидений

Критерий	Dreamovы	Awoken	Lucid Dreamer
Анализ символов/паттернов	–	–	–
Визуализация статистики (графики)	± (базовая)	–	–
Наличие тегов / категорий	+	+	–
Экспорт данных (PDF/CSV)	–	–	–
Бесплатная версия (базовый функционал)	± (ограничена)	+	–
Голосовой ввод / аудиодневник	–	–	+
Веб-доступ (с любого устройства)	–	–	–
Наличие ИИ (генерация иллюстраций / анализ)	–	–	–

1.2 Анализ инструментов разработки

Для реализации веб-приложения «Книга снов» необходимо выбрать оптимальный стек технологий, обеспечивающий производительность, удобство разработки и достаточный уровень безопасности. Проведём сравнительный анализ наиболее популярных инструментов для создания веб-приложений аналогичного класса.

Сравнение технологий бэкенда. Бэкенд-часть отвечает за обработку запросов, взаимодействие с базой данных, аутентификацию пользователей и бизнес-логику. Рассмотрим три распространённых подхода: классический синхронный (PHP), асинхронный на Python (Flask/Django) и корпоративный на Java (Spring). Сравнение представлено в таблице 1.2.

Таблица 1.2 – Сравнительный анализ технологий бэкенда

Критерий	PHP (Laravel)	Python (Flask/Django)	Java (Spring)
Порог входа	низкий	низкий	высокий
Скорость разработки	высокая	высокая	средняя
Производительность	средняя	средняя	высокая
Экосистема библиотек	широкая	очень широкая	широкая

Анализ таблицы показывает, что Java (Spring) избыточна для небольшого веб-приложения; PHP (Laravel) также применим, однако экосистема Python предоставляет более удобные инструменты для анализа текстов (библиотеки NLTK, rumorphy2) и быстрого прототипирования.

Сравнение технологий фронтенда. Фронтенд-часть определяет пользовательский интерфейс и взаимодействие с сервером. На рынке доминируют три подхода: классический (HTML/CSS/JS без фреймворков), React и Vue.js. Сравнение приведено в таблице 1.3.

Таблица 1.3 – Сравнительный анализ технологий фронтенда

Критерий	Чистый JS	React	Vue.js
Порог входа	низкий	средний	низкий
Скорость разработки	низкая	высокая	высокая
Интерактивность UI	ручная реализация	высокая	высокая
Экосистема	базовая	огромная	растущая
Подходит для анализа данных	–	+	+

Анализ показывает, что использование чистого JavaScript без фреймворков приводит к значительному росту объёма кода при реализации динамического интерфейса (графики, формы, асинхронные запросы). React и Vue.js одинаково хорошо подходят для визуализации статистики, однако React обладает более широкой экосистемой.

Выбор стека технологий для проекта. На основе проведённого сравнительного анализа для реализации веб-приложения «Книга снов» выбран следующий стек технологий:

- **Бэкенд:** язык Python, микрофреймворк Flask. Python обеспечивает простоту написания серверной логики, широкий выбор библиотек для анализа текстов и аутентификации. Flask даёт гибкость, необходимую для небольшого приложения с чётко определённой функциональностью.
- **База данных:** SQLite. Простейшая СУБД, отлично подходящая для хранения записей снов, пользовательских данных и статистики.
- **Фронтенд:** HTML5, CSS3, Bootstrap 5, чистый JavaScript. Выбран классический подход без тяжёлых фреймворков, так как приложение не требует сложной клиентской маршрутизации, а Bootstrap 5 обеспечивает адаптивный интерфейс «из коробки».
- **Дополнительные библиотеки:** Flask-SQLAlchemy (ORM), Flask-Login (аутентификация), Fetch API (асинхронные запросы).

Таким образом, выбранный стек технологий полностью покрывает функциональные требования к приложению «Книга снов» и соответствует современным практикам веб-разработки.

1.3 Формулировка цели и задач

На основе проведенного анализа инструментов (см. подраздел 1.2) и выявленных пробелов в существующих продуктах, была сформулирована цель исследования.

Цель работы — создание веб-приложения «Книга снов», обеспечивающего ведение личного дневника сновидений с возможностью статистического анализа записей.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести сравнительный анализ существующих решений для ведения дневников сновидений и обосновать выбор технологического стека;
2. Разработать архитектуру клиент-серверного приложения и спроектировать структуру базы данных для хранения записей снов и пользовательских данных;
3. Реализовать программные модули для аутентификации пользователей и разграничения прав доступа по ролям;
4. Реализовать CRUD-интерфейс для управления записями снов и агрегации статистических данных;
5. Разработать модуль экспорта дневника в CSV и провести тестирование разработанной системы.

ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории

Для определения функциональных требований к веб-приложению «Книга снов» был проведён анализ целевой аудитории. В отличие от корпоративных систем, разрабатываемое приложение ориентировано на частных пользователей с различными внутренними мотивациями. На основе исследования существующих решений выделены три ключевых сегмента пользователей.

Первый сегмент — психологически ориентированные пользователи.

К этой группе относятся люди, которые ведут дневник сновидений для самоанализа, отслеживания эмоционального состояния или в рамках психотерапии. Для них ценность приложения заключается в способности автоматически выявлять повторяющиеся символы и паттерны, а также визуализировать динамику эмоционального фона.

Второй сегмент — творческие люди (писатели, художники, сценаристы). Они используют сны как источник образов и вдохновения. Для этой аудитории важна эстетическая составляющая: оформление в виде «книги», возможность генерации иллюстраций к снам с помощью ИИ, а также удобная навигация по архиву записей.

Третий сегмент — практикующие осознанные сновидения. Это пользователи, которые целенаправленно работают над развитием навыка осознания себя во сне. Их ключевые потребности — отслеживание частоты снов, фиксация прогресса, календарь с визуальной индикацией.

Для более детального понимания потребностей каждого сегмента был применён метод персонажей (Personas).

Персонаж 1: Анна, 32 года, психолог. Анна ведёт дневник сновидений уже два года, используя обычные заметки в телефоне. Она хочет отслеживать, какие символы (вода, огонь, падение, полёт) повторяются в её снах в периоды стресса. Главная «боль»: существующие приложения не дают аналитики, приходится просматривать записи вручную. Приоритетные функции: график эмоций, облако тегов, экспорт статистики в CSV.

Персонаж 2: Дмитрий, 25 лет, художник. Дмитрий часто видит яркие сны и хочет сохранять их не только текстом, но и визуально. Ему интересна функция автоматической генерации иллюстрации к сну через нейросеть. Также он хочет красиво оформить архив своих снов. Приоритетные функции:

генерация картинки по тексту сна, экспорт в CSV как «книга снов».

Персонаж 3: Елена, 29 лет, практик осознанных сновидений. Елена уже три года тренирует осознанность во сне и ведёт подробный дневник. Ей важно отслеживать частоту снов по дням, отмечать, в какие ночи были осознанные сновидения. Елена не хочет тратить время на ручное форматирование записей, ей нужен быстрый ввод и наглядный календарь. Приоритетные функции: календарь снов, статистика частоты, быстрая регистрация сна.

Помимо обычных пользователей, в системе предусмотрена роль **администратора**. Администратор выполняет обслуживающие функции: управление справочником символов (добавление, редактирование, удаление), блокировка пользователей, просмотр общей статистики.

Проведённый анализ целевой аудитории позволяет сформулировать следующие выводы: приложение должно поддерживать три основные группы пользователей, каждая из которых имеет свои приоритетные функции; интерфейс не должен быть перегружен, но обязан предоставлять доступ ко всем ключевым функциям; роль администратора должна быть выделена отдельно с расширенными правами.

2.2 Описание функциональности приложения

Функциональные возможности системы определяются через взаимодействие пользователя с системой. На основе анализа целевой аудитории (см. раздел 2.1) были сформулированы следующие пользовательские истории (User Stories).

Пользовательская история для психологически ориентированного пользователя: «Как психологически ориентированный пользователь, я хочу автоматически получать статистику по повторяющимся символам в моих снах и график эмоционального фона, чтобы отслеживать своё психоэмоциональное состояние в динамике».

Критерии приёмки:

- система выделяет из текста сна ключевые символы;
- доступен график эмоционального фона по датам;
- облако тегов визуализирует самые частые символы.

Пользовательская история для творческого пользователя: «Как творческий пользователь, я хочу прикреплять к снам изображения и экспортировать дневник в PDF, чтобы сохранять визуальные образы снов».

Критерии приёмки:

- загрузка изображений к любому сну;
- галерея снов с миниатюрами;
- экспорт всей книги снов в CSV с текстом и картинками.

Пользовательская история для практикующего осознанные сновидения: «Как практикующий осознанные сновидения, я хочу быстро вносить записи и видеть календарь с отмеченными днями снов, чтобы отслеживать прогресс».

Критерии приёмки:

- минимальная форма создания сна;
- календарь с цветовой индикацией дней со снами;
- переход по клику на день к записи сна.

На рисунке 2.1 представлена диаграмма вариантов использования (Use Case), обобщающая все сценарии.



Рисунок 2.1 – Диаграмма вариантов использования системы «Книга снов»

2.3 Проектирование модели данных

При выборе архитектуры базы данных был проведен сравнительный анализ эффективности различных решений, опирающийся на данные о производительности современных СУБД.

В таблице 2.1 представлено сравнение наиболее популярных реляционных СУБД.

Таблица 2.1 – Сравнительный анализ СУБД для задач «Книги снов»

Критерий	PostgreSQL	MySQL	SQLite
Полнотекстовый поиск по тексту	++ (встроенный, мощный)	+ (есть, но слабее)	– (нет)
Хранение и поиск по JSON	++ (отличная поддержка)	+ (базовая)	– (нет)
Поддержка массивов (для хранения символов)	++ (есть тип array)	– (нет)	– (нет)
Работа с большим количеством записей (1000+)	++	++	– (тормозит)
Поддержка внешних ключей и ссылочной целостности	++	++ (только InnoDB)	+
Простота развёртывания для курсовой	средняя	средняя	высокая
Бесплатная лицензия	Да	Да	Да

На основе проведённого анализа для реализации веб-приложения «Книга снов» выбрана СУБД **SQLite**. Данный выбор обусловлен следующими преимуществами:

Структура базы данных. Спроектированная база данных состоит из девяти таблиц, обеспечивающих хранение пользовательской информации, записей снов, символов для анализа, а также данных для статистики и ИИ-генерации изображений.

Основные таблицы:

- `users` — хранит учётные записи пользователей (логин, хеш пароля, роль, аватар, дата регистрации);
- `dreams` — содержит записи снов (заголовок, текст, дата сна, оценка настроения, внешний ключ на пользователя);
- `symbols` — справочник символов для анализа текстов (вода, огонь, полёт и т.д.);
- `dream_symbols` — связующая таблица для реализации отношения «многие ко многим» между снами и символами.
- `user_images` — хранит изображения, загруженные пользователем самостоятельно (не через ИИ-генерацию).

Таблицы для аналитики и кэширования:

- `statistics` — хранит кэшированную статистику в формате JSON для быстрого отображения;
- `emotion_stats` — содержит сводки по эмоциональному фону (дневные, недельные, месячные);
- `symbol_stats` — хранит частоту встречаемости символов по периодам.

Таблицы для ИИ-генерации:

- `generation_requests` — фиксирует запросы к ИИ (промпт, статус выполнения, временная метка);
- `generated_images` — хранит URL сгенерированных изображений и связь с соответствующим сном и запросом.

ER-диаграмма, визуализирующая описанные сущности и связи, представлена на рисунке 2.2.

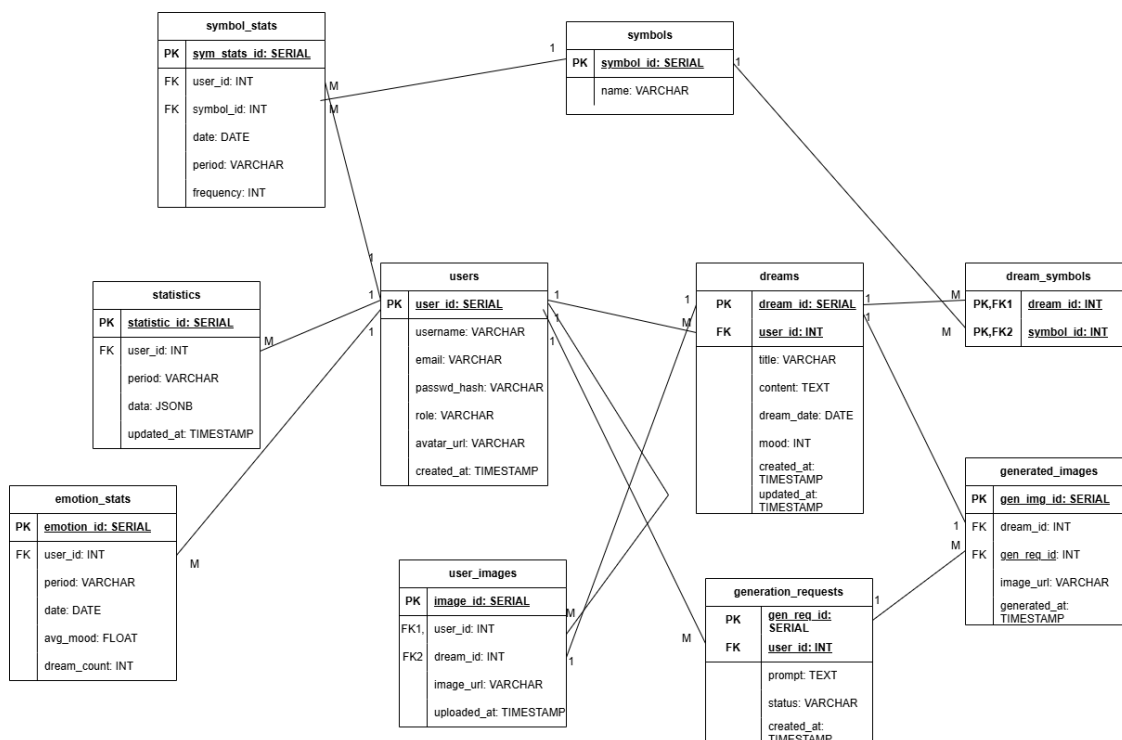


Рисунок 2.2 – ER-диаграмма базы данных «Книга снов»

2.4 Разработка макетов страниц

На основе спроектированной функциональности и модели данных были разработаны макеты основных страниц веб-приложения «Книга снов». Макеты выполнены в стиле вайрфреймов (wireframes), демонстрирующих расположение ключевых элементов интерфейса.

Макет страницы входа и регистрации. На рисунке 2.3 представлен макет главной страницы. Страница содержит две формы: «Вход» для существующих пользователей (поля Email и Пароль, кнопка «Войти») и «Регистрация» для новых пользователей (поля Логин, Email, Пароль, кнопка «Зарегистрироваться»). Панель навигации отсутствует, так как пользователь ещё не авторизован.



Книга снов

Регистрация

Логин

Email

Пароль

Зарегистрироваться

Вход

Email

Пароль

Войти

Подвал сайта

Рисунок 2.3 – Макет страницы входа и регистрации

Макет страницы «Моя книга снов». На рисунке 2.4 представлен макет личного кабинета пользователя. В верхней части расположена панель навигации с пунктами: «Моя книга», «Новый сон», «Статистика», «Профиль». Основная область содержит галерею карточек снов. Каждая карточка включает миниатюру изображения, заголовок, дату сна и оценку настроения. В правом верхнем углу находится кнопка «Новый сон» для быстрого создания записи.



Моя книга

Новый сон

Статистика

Профиль

+ Новый сон

Путешествие с котом



12.04.2026

★☆☆☆☆

Полет во сне



12.04.2026

★☆☆☆☆

Поедание конфет



12.04.2026

★☆☆☆☆

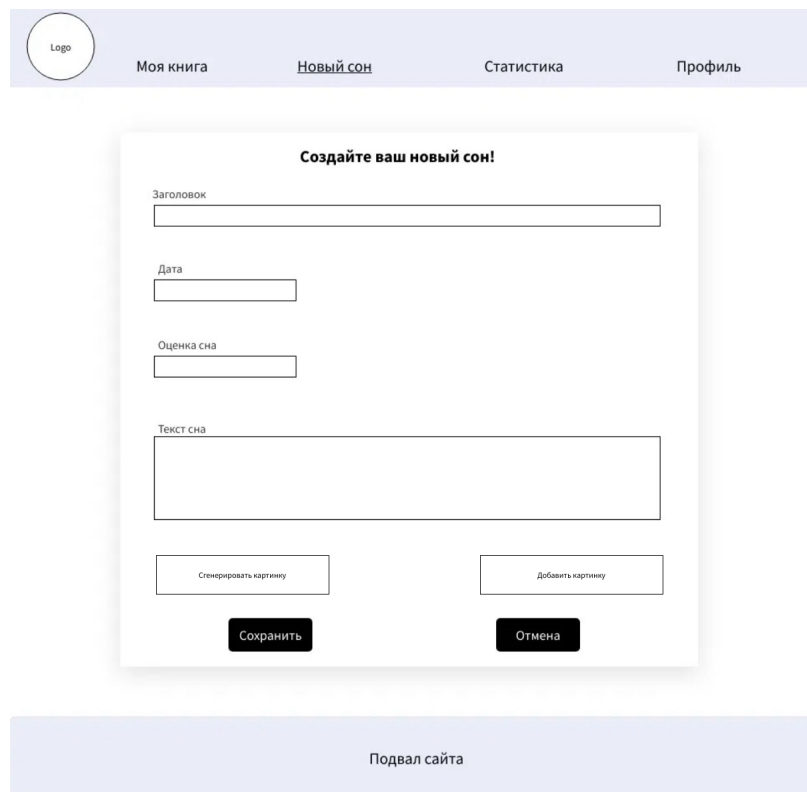
Статистика

Настройки

Подвал сайта

Рисунок 2.4 – Макет страницы «Моя книга снов»

Макет страницы создания и редактирования сна. На рисунке 2.5 представлен макет формы создания нового сна. Страница содержит следующие поля: заголовок, дата сна, оценка настроения, текст сна (многострочное поле). Внизу формы расположены кнопки «Сохранить» и «Отмена». Предусмотрена кнопка ИИ-генерации изображения, а также самостоятельного добавления картинки пользователем.



Logo

Моя книга Новый сон Статистика Профиль

Создайте ваш новый сон!

Заголовок

Дата

Оценка сна

Текст сна

Сгенерировать картинку

Добавить картинку

Сохранить Отмена

Подвал сайта

Рисунок 2.5 – Макет страницы создания и редактирования сна

Макет страницы просмотра одного сна. На рисунке 2.6 представлен макет детальной страницы сна. Страница разделена на две колонки: слева отображается изображение, справа — метаданные: дата сна, оценка настроения, список распознанных символов. Ниже располагается полный текст сна. Под текстом находятся кнопки «Редактировать», «Удалить» и «Перегенерировать картинку».

Logo

Моя книга

Новый сон

Статистика

Профиль

Заголовок сна

Дата

Настроение

Символы

Текст сна

Редактировать

Удалить

Перегенерировать картинку

Назад к книге снов

Подвал сайта

Рисунок 2.6 – Макет страницы просмотра одного сна

Макет страницы статистики и аналитики.

На рисунке 2.7 представлен макет страницы статистики. Страница содержит график эмоционального фона (линейный график изменения средней оценки настроения по датам), облако тегов (размер шрифта символа пропорционален частоте его встречаемости), блок «Топ-5 символов» и блок общей статистики (количество снов, средняя оценка, самый продуктивный месяц). В нижней части расположены кнопки экспорта статистики и всей книги снов в CSV.

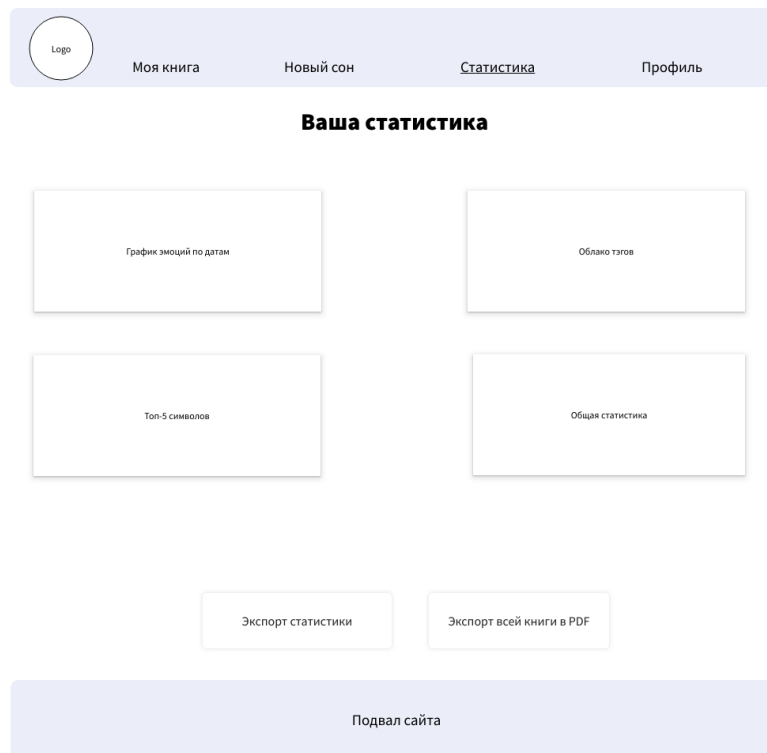


Рисунок 2.7 – Макет страницы статистики и аналитики

Макет страницы профиля пользователя.

На рисунке 2.8 представлен макет страницы профиля. В левой части отображается аватар пользователя, в правой — личные данные: имя пользователя, email, дата регистрации и роль. Ниже расположен блок краткой статистики (количество снов, средняя оценка настроения, самый частый символ). Внизу страницы расположены кнопка «Выйти из аккаунта».

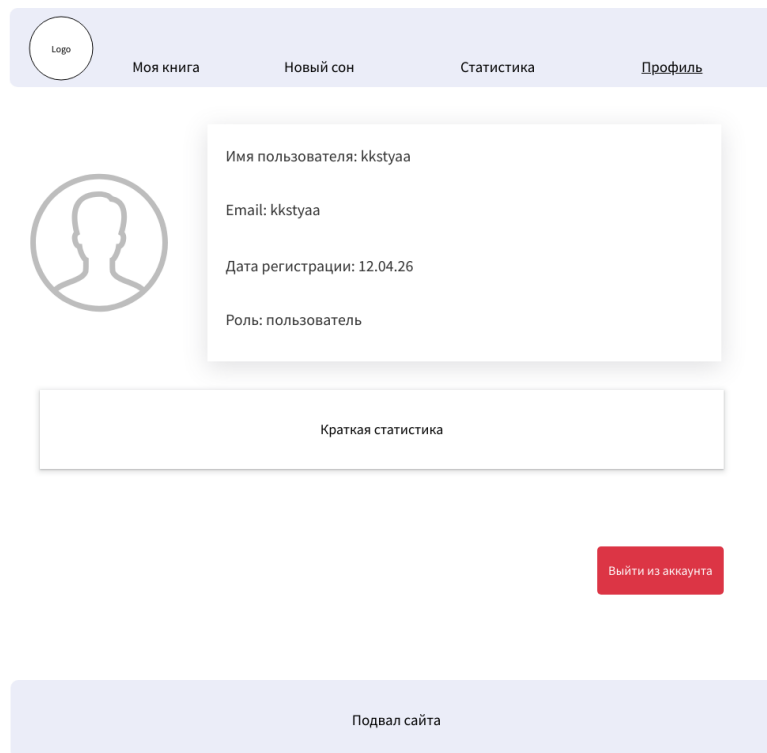


Рисунок 2.8 – Макет страницы профиля пользователя

Разработанные макеты охватывают весь ключевой функционал веб-приложения и служат основой для последующей программной реализации клиентской части.

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Модели базы данных

В соответствии с задачей 3.1, была спроектирована объектно-реляционная модель базы данных с использованием библиотеки **Flask-SQLAlchemy**. Модели реализованы в виде Python-классов, отображаемых на таблицы реляционной базы данных SQLite. Пример модели пользователя представлен в листинге 1.

Листинг 1 – Модель пользователя

```
1 class User(db.Model, UserMixin):
2     __tablename__ = 'users'
3
4     user_id = db.Column(db.Integer, primary_key=True)
5     username = db.Column(db.String(100), unique=True,
6         nullable=False)
7     email = db.Column(db.String(255), unique=True, nullable=
8         False)
9     password_hash = db.Column(db.String(255), nullable=False
10    )
11     role = db.Column(db.String(20), default='user')
12     avatar_url = db.Column(db.String(500), nullable=True)
13     created_at = db.Column(db.DateTime, default=datetime.now
14    )
15
16     dreams = db.relationship('Dream', backref='author', lazy
17    =True)
18     statistics = db.relationship('Statistic', backref='user
19    ', lazy=True)
20     emotion_stats = db.relationship('EmotionStat', backref='
21    user', lazy=True)
22     symbol_stats = db.relationship('SymbolStat', backref='
23    user', lazy=True)
24     generation_requests = db.relationship('GenerationRequest
25    ', backref='user', lazy=True)
26     user_images = db.relationship('UserImage', backref='user
27    ', lazy=True)
```

Листинг 2 демонстрирует модель сна, связанную с пользователем и символами.

Листинг 2 – Модель сна

```
1 class Dream(db.Model):
```

```

2     __tablename__ = 'dreams'
3
4     dream_id = db.Column(db.Integer, primary_key=True)
5     user_id = db.Column(db.Integer, db.ForeignKey('users.
user_id'), nullable=False)
6     title = db.Column(db.String(200), nullable=False)
7     content = db.Column(db.Text, nullable=False)
8     dream_date = db.Column(db.Date, nullable=False)
9     mood = db.Column(db.Integer)
10    created_at = db.Column(db.DateTime, default=datetime.now
)
11    updated_at = db.Column(db.DateTime, default=datetime.now
, onupdate=datetime.now)
12
13    # связи
14    dream_symbols = db.relationship('DreamSymbol', backref='
dream', lazy=True)
15    generated_image = db.relationship('GeneratedImage',
backref='dream', uselist=False)
16    user_image = db.relationship('UserImage', backref='dream
', uselist=False)

```

Всего в рамках проекта было реализовано 10 сущностей, что превышает минимальное требование (не менее 3).

3.2 Аутентификация и личные кабинеты

Согласно задаче 3.2, в системе реализована ролевая модель доступа с использованием библиотеки **Flask-Login**. Для обеспечения безопасности пароли пользователей хранятся в базе данных в хешированном виде (функция `generate_password_hash` из модуля `werkzeug.security`).

Листинг 3 демонстрирует логику аутентификации пользователя.

Листинг 3 – Обработчик входа в систему

```

1 @bp.route('/login', methods=['GET', 'POST'])
2 def login():
3     if current_user.is_authenticated:
4         return redirect(url_for('index'))
5
6     if request.method == 'POST':
7         username = request.form.get('login')
8         password = request.form.get('password')
9         remember = request.form.get('remember')

```

```

10
11         user = User.query.filter_by(username=username).first
12         ()
13         if user and user.check_password(password):
14             login_user(user, remember=bool(remember))
15             flashВы(' успешно вошли в систему!', 'success')
16             return redirect(url_for('index'))
17             flashНеверный(' логин или пароль', 'danger')
18
19     return render_template('auth/login.html')

```

Для создания новой учётной записи реализован маршрут регистрации (листинг 4). Перед сохранением выполняется проверка уникальности логина и email, а также совпадения паролей.

Листинг 4 – Обработчик регистрации пользователя

```

1 @bp.route('/register', methods=['GET', 'POST'])
2 def register():
3     if current_user.is_authenticated:
4         return redirect(url_for('index'))
5
6     if request.method == 'POST':
7         username = request.form.get('login')
8         email = request.form.get('email')
9         password = request.form.get('password')
10        confirm = request.form.get('confirm_password')
11
12        if password != confirm:
13            flashПароли(' не совпадают', 'danger')
14            return redirect(url_for('auth.register'))
15
16        if User.query.filter_by(username=username).first():
17            flashПользователь(' с таким логином уже существует', '
18            danger')
19            return redirect(url_for('auth.register'))
20
21        if User.query.filter_by(email=email).first():
22            flashПользователь(' с таким email уже существует', '
23            danger')
24            return redirect(url_for('auth.register'))
25
26        user = User(username=username, email=email)
27        user.set_password(password)

```



```

26         db.session.add(user)
27         db.session.commit()
28
29         login_user(user)
30         flashРегистрация(' прошла успешно!', 'success')
31         return redirect(url_for('index'))
32
33     return render_template('auth/register.html')

```

После успешной аутентификации пользователь перенаправляется в личный кабинет (страница профиля), где отображается его информация и краткая статистика. Пример шаблона страницы профиля представлен в листинге 5.

Листинг 5 – Шаблон страницы профиля (profile.html)

```

1 <div class="profile-container">
2     <!-- Основная карточка профиля -->
3     <div class="profile-card">
4         <div class="row align-items-center">
5             <!-- Аватар -->
6             <div class="col-md-3 profile-avatar-section">
7                 <div class="avatar-circle">
8                     {% if current_user.avatar_url %}
9                         
11                     {% else %}
12                         <span class="avatar-text">{{
current_user.username[0]|upper }}</span>
13                     {% endif %}
14                 </div>
15                 <form method="POST" action="{{ url_for('auth
.upload_avatar') }}" enctype="multipart/form-data" class
="mt-2">
16                     <input type="file" name="avatar" accept
="image/*" id="avatarInput" style="display: none;"
onchange="this.form.submit()">
17                     <label for="avatarInput" class="avatar-
edit" style="cursor: pointerСменить;"> аватар</label>
18                 </form>
19             </div>
20             <!-- Информация о пользователе -->

```

```

21         <div class="col-md-9 profile-info-section">
22             <h1 class="profile-username">{{ current_user
23                 .username }}</h1>
24             <span class="profile-role">{{ current_user .
25                 role|defaultПользователь('') }}</span>
26
27             <div class="profile-detail">
28                 <span class="profile-detail-text">{{
29                     current_user.email }}</span>
30             </div>
31             <div class="profile-detail">
32                 <span class="profile-detail-text
33                     Зарегистрирован">: {{ current_user.created_at.strftime('%d
34                     .%m.%Y') }}</span>
35             </div>
36         </div>
37     </div>
38 </div>

```

Для выхода из системы используется маршрут `/logout`, который завершает сессию пользователя и перенаправляет на главную страницу. Ролевая модель доступа реализована через поле `role` в модели пользователя и декоратор `@admin_required`, ограничивающий доступ к административным функциям.

3.3 CRUD-интерфейс для управления снами

В рамках задачи 3.3 реализован полноценный CRUD-интерфейс для управления записями снов. Все операции доступны только авторизованным пользователям и ограничены их собственными данными.

Листинг 6 иллюстрирует создание новой записи сна.

Листинг 6 – Создание сна

```

1 @bp.route('/new', methods=['GET', 'POST'])
2 @login_required
3 def new():
4     if request.method == 'POST':
5         title = request.form.get('title')
6         content = request.form.get('content')
7         dream_date = datetime.strptime(
8             request.form.get('dream_date'), '%Y-%m-%d'
9         ).date()

```

```

10         mood = request.form.get('mood', type=int)
11
12         dream = Dream(
13             user_id=current_user.user_id,
14             title=title,
15             content=content,
16             dream_date=dream_date,
17             mood=mood
18         )
19         db.session.add(dream)
20         db.session.commit()
21         flashСон(' успешно добавлен!', 'success')
22         return redirect(url_for('dreams.index'))
23     return render_template('dreams/form.html', dream=None)

```

Просмотр списка снов (листинг 7) реализован с пагинацией — на одной странице отображается не более 9 записей.

Листинг 7 – Просмотр списка снов

```

1 @bp.route('/')
2 @login_required
3 def index():
4     page = request.args.get('page', 1, type=int)
5     per_page = 9
6
7     pagination = Dream.query.filter_by(
8         user_id=current_user.user_id
9     ).order_by(Dream.dream_date.desc()).paginate(
10         page=page, per_page=per_page, error_out=False
11     )
12     dreams = pagination.items
13
14     return render_template('dreams/index.html',
15                             dreams=dreams, pagination=
16                             pagination)

```

Просмотр детальной страницы одного сна (листинг 8) выполняется по его идентификатору с проверкой прав доступа.

Листинг 8 – Просмотр одного сна

```

1 @bp.route('/<int:dream_id>')
2 @login_required
3 def show(dream_id):

```

```

4     dream = Dream.query.get_or_404(dream_id)
5     if dream.user_id != current_user.user_id:
6         flashV(' вас нет доступа к этому сну', 'danger')
7         return redirect(url_for('dreams.index'))
8     return render_template('dreams/show.html', dream=dream)

```

Листинг 9 демонстрирует редактирование существующей записи сна.

Листинг 9 – Редактирование сна

```

1 @bp.route('/<int:dream_id>/edit', methods=['GET', 'POST'])
2 @login_required
3 def edit(dream_id):
4     dream = Dream.query.get_or_404(dream_id)
5     if dream.user_id != current_user.user_id:
6         flashV(' вас нет доступа к этому сну', 'danger')
7         return redirect(url_for('dreams.index'))
8
9     if request.method == 'POST':
10        dream.title = request.form.get('title')
11        dream.content = request.form.get('content')
12        dream.dream_date = datetime.strptime(
13            request.form.get('dream_date'), '%Y-%m-%d'
14        ).date()
15        dream.mood = request.form.get('mood', type=int)
16        dream.updated_at = datetime.now()
17        db.session.commit()
18
19        flashСон(' успешно обновлён!', 'success')
20        return redirect(url_for('dreams.show', dream_id=
21            dream.dream_id))
22    return render_template('dreams/form.html', dream=dream)

```

Листинг 10 показывает реализацию удаления записи сна.

Листинг 10 – Удаление сна

```

1 @bp.route('/<int:dream_id>/delete', methods=['GET', 'POST'])
2 @login_required
3 def delete(dream_id):
4     Удаление"""" сна""""
5     dream = Dream.query.get_or_404(dream_id)
6     if dream.user_id != current_user.user_id:
7         flashV(' вас нет доступа к этому сну', 'danger')
8         return redirect(url_for('dreams.index'))

```

```

9
10     db.session.delete(dream)
11     db.session.commit()
12     update_all_stats(current_user.user_id)
13     flashСон(' удалён', 'success')
14     return redirect(url_for('dreams.index'))

```

Таким образом, реализованы все четыре базовые операции CRUD: создание, чтение (список и детальный просмотр), редактирование и удаление. Каждая операция сопровождается проверкой прав доступа (только владелец сна может его изменять или удалять) и всплывающими уведомлениями о результате выполнения.

3.4 Загрузка файлов

Согласно задаче 3.4, в приложении реализована функция загрузки файлов двух типов: аватар пользователя и иллюстрации к снам. Файлы сохраняются на сервер с генерацией уникального имени для предотвращения коллизий. Для загрузки аватара используется отдельный маршрут `/auth/upload_avatar` (листинг 11). Форма загрузки расположена на странице профиля и отправляется автоматически после выбора файла.

Листинг 11 – Загрузка аватара пользователя

```

1 @bp.route('/upload_avatar', methods=['POST'])
2 @login_required
3 def upload_avatar():
4     if 'avatar' not in request.files:
5         flashФайл(' не выбран', 'danger')
6         return redirect(url_for('profile'))
7
8     file = request.files['avatar']
9     filename = save_file(file, current_app.config['
AVATAR_FOLDER'])
10
11     if filename:
12         current_user.avatar_url = f'/uploads/avatars/{
filename}'
13         db.session.commit()
14         flashАватар(' успешно обновлён!', 'success')
15     else:
16         flashНедопустимый(' формат файла', 'danger')
17

```

```
18     return redirect(url_for('profile'))
```

Функция `save_file` (листинг 12) является универсальной и используется как для аватаров, так и для иллюстраций к снам.

Листинг 12 – Универсальная функция сохранения файла

```
1 def save_file(file, folder):
2     """ Сохраняет файл и возвращает его имя """
3     if not file or file.filename == '':
4         return None
5
6     if not allowed_file(file.filename):
7         return None
8
9     # Генерируем уникальное имя файла
10    ext = file.filename.rsplit('.', 1)[1].lower()
11    filename = f"{uuid.uuid4().hex}.{ext}"
12
13    filepath = os.path.join(folder, filename)
14    file.save(filepath)
15
16    return filename
```

Загрузка изображений к сну реализована непосредственно в форме создания и редактирования сна. При отправке формы файл обрабатывается и сохраняется в папку `uploads/dream_images/`. Листинг 13 демонстрирует фрагмент кода обработки загруженного изображения при создании сна.

Листинг 13 – Загрузка изображения к сну

```
1 if 'user_image' in request.files:
2     file = request.files['user_image']
3     if file and file.filename:
4         filename = save_file(file,
5                               current_app.config['DREAM_IMAGES_FOLDER'])
6         if filename:
7             user_image = UserImage(
8                 dream_id=dream.dream_id,
9                 user_id=current_user.user_id,
10                image_url=f'/uploads/dream_images/{filename}'
11            )
12            db.session.add(user_image)
```

Для доступа к загруженным файлам реализован специальный маршрут (листинг 14), который возвращает файл из директории uploads.

Листинг 14 – Маршрут для раздачи загруженных файлов

```
1 @app.route('/uploads/<path:filename>')
2     def uploaded_file(filename):
3         return send_from_directory(app.config['UPLOAD_FOLDER'], filename)
```

В конфигурационном файле заданы ограничения на типы и размер загружаемых файлов (листинг 15).

Листинг 15 – Настройки загрузки файлов в config.py

```
1 UPLOAD_FOLDER = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'uploads')
2 AVATAR_FOLDER = os.path.join(UPLOAD_FOLDER, 'avatars')
3 DREAM_IMAGES_FOLDER = os.path.join(UPLOAD_FOLDER, 'dream_images')
4 MAX_CONTENT_LENGTH = 16 * 1024 * 1024 # 16MB
5 ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif', 'webp'}
```

Таким образом, в приложении реализована полноценная загрузка файлов обоих типов с проверкой допустимых расширений, уникальной генерацией имён и корректной раздачей через статические маршруты.

3.5 Анализ текста сна

В соответствии с задачей 3.5, реализован модуль автоматического распознавания символов в тексте сна. Алгоритм ищет в тексте ключевые слова, соответствующие записям из справочника символов, и сохраняет связи в таблице dream_symbols.

Листинг 16 показывает основную логику анализа.

Листинг 16 – Анализ текста и сохранение символов

```
1 def analyze_and_save_symbols(dream_id, content):
2     if not content:
3         return
4
5     # Получаем все символы из БД
6     all_symbols = Symbol.query.all()
7
```

```

8     if not all_symbols:
9         return
10
11     found_symbols = []
12     content_lower = content.lower()
13
14     for symbol in all_symbols:
15         # Ищем слово целиком не( часть другого слова)
16         pattern = r'\b' + re.escape(symbol.name.lower()) + r
17         '\b'
18         if re.search(pattern, content_lower):
19             found_symbols.append(symbol)
20
21     # Удаляем старые связи
22     DreamSymbol.query.filter_by(dream_id=dream_id).delete()
23
24     # Добавляем новые
25     for symbol in found_symbols:
26         dream_symbol = DreamSymbol(dream_id=dream_id,
27                                     symbol_id=symbol.symbol_id)
28         db.session.add(dream_symbol)
29
30     db.session.commit()

```

Данная функция вызывается при создании и редактировании сна, обеспечивая актуальность связей между снами и символами.

3.6 Агрегация данных и статистика

Согласно задаче 3.6, в приложении реализован модуль агрегации данных для построения статистики. Расчёт выполняется на лету с использованием агрегатных функций SQLAlchemy, а также кэшируется в таблице `statistics` для ускорения отображения.

Листинг 17 демонстрирует расчёт количества снов и средней оценки.

Листинг 17 – Расчёт основных статистических показателей

```

1 @bp.route('/')
2 @login_required
3 def index():
4     user_id = current_user.user_id
5
6     total_dreams = Dream.query.filter_by(user_id=user_id).
7     count()

```



```

7     avg_mood = db.session.query(
8         func.avg(Dream.mood)
9     ).filter_by(user_id=user_id).scalar()
10
11     monthly_counts = db.session.query(
12         func.strftime('%Y-%m', Dream.dream_date).label('
month'),
13         func.count(Dream.dream_id).label('count')
14     ).filter_by(user_id=user_id).group_by('month').all()
15
16     return render_template('statistics/index.html',
17                           total_dreams=total_dreams,
18                           avg_mood=avg_mood,
19                           monthly_counts=monthly_counts)

```

Для визуализации данных на фронтенде используется библиотека **Chart.js**, позволяющая строить столбчатые и линейные графики динамики снов и эмоционального фона.

3.7 Экспорт в CSV

В рамках задачи 3.7 реализован модуль экспорта данных в формат CSV. Пользователь может экспортировать как сводную статистику (общая статистика + топ-символы), так и полный список всех своих снов. Листинг 18 демонстрирует экспорт книги снов.

Листинг 18 – Экспорт списка снов в CSV

```

1 @bp.route('/export/dreams')
2 @login_required
3 def export_dreams():
4     Экспорт"" все снов книги() в CSV""
5     user_id = current_user.user_id
6     dreams = Dream.query.filter_by(user_id=user_id).order_by
(Dream.dream_date.desc()).all()
7
8     si = StringIO()
9     cw = csv.writer(si, delimiter=';')
10    cw.writerow(['ID', Заголовок'', Дата' сна', Настроение'
(1-5)', Текст' сна', Дата' создания'])
11
12    for dream in dreams:
13        cw.writerow([
14            dream.dream_id,

```

```

15         dream.title,
16         dream.dream_date.strftime('%d.%m.%Y'),
17         dream.mood or '',
18         dream.content[:200] + '...' if len(dream.content
) > 200 else dream.content,
19         dream.created_at.strftime('%d.%m.%Y %H:%M')
20     ])
21
22     output = si.getvalue()
23     try:
24         output_bytes = output.encode('windows-1251')
25     except:
26         output_bytes = output.encode('utf-8-sig')
27
28     return Response(
29         output_bytes,
30         mimetype='text/csv',
31         headers={'Content-Disposition': 'attachment;filename
=dreams_export.csv'})
32 )

```

Экспорт статистики выполняется аналогично, но содержит агрегированные данные.

3.8 Панель администратора

Согласно задаче 3.8, реализована панель администратора с расширенными правами. Доступ к панели ограничен декоратором `@admin_required`, который проверяет роль пользователя. Листинг 19 показывает пример управления символами.

Листинг 19 – Управление символами в админ-панели

```

1 @bp.route('/symbols')
2 @login_required
3 @admin_required
4 def symbols():
5     symbols_list = Symbol.query.order_by(Symbol.name).all()
6     return render_template('admin/symbols.html', symbols=
symbols_list)
7
8 @bp.route('/symbols/new', methods=['GET', 'POST'])
9 @login_required
10 @admin_required

```

```

11 def new_symbol():
12     if request.method == 'POST':
13         name = request.form.get('name', '').strip()
14         if Symbol.query.filter_by(name=name).first():
15             flashСимвол(' уже существует', 'danger')
16             return redirect(url_for('admin.new_symbol'))
17
18         symbol = Symbol(name=name)
19         db.session.add(symbol)
20         db.session.commit()
21         flashСимвол(' добавлен', 'success')
22         return redirect(url_for('admin.symbols'))
23     return render_template('admin/symbol_form.html', symbol=
None)

```

Администратор имеет возможность просматривать список всех пользователей, изменять их роли, удалять пользователей, а также добавлять, редактировать и удалять символы из справочника. Репозиторий проекта доступен по адресу: https://github.com/kkstyaa/dream-book_project

ОФОРМЛЕНИЕ ИТОГОВ РАБОТЫ

4.1 Git-репозиторий

В ходе выполнения курсового проекта велась работа с системой контроля версий **Git**. Все этапы разработки фиксировались в локальном репозитории, затем синхронизировались с удалённым хостингом **GitHub**. Это позволило отслеживать изменения кода, возвращаться к предыдущим версиям и обеспечивать резервное копирование проекта.

Репозиторий проекта доступен по адресу:

https://github.com/kkstyaa/dream-book_project

Структура репозитория включает:

- папку `app/` — весь исходный код веб-приложения (модели, маршруты, шаблоны, статические файлы);
- файлы конфигурации (`config.py`, `requirements.txt`);
- скрипт инициализации базы данных (`init_db.py`);
- точку входа в приложение (`run.py`);
- документацию к проекту (папка `note/`).

4.2 Деплой на хостинг

Разработанное веб-приложение было развёрнуто на **PythonAnywhere** — хостинге, специализирующемся на размещении Python-приложений. Бесплатный тариф платформы предоставляет одно веб-приложение с возможностью использования базы данных SQLite, что полностью покрывает потребности проекта.

Процесс деплоя включал следующие шаги:

1. Регистрация аккаунта на PythonAnywhere;
2. Загрузка исходного кода проекта через веб-интерфейс;
3. Установка зависимостей из `requirements.txt`;
4. Запуск скрипта инициализации базы данных `init_db.py`;

5. Настройка статических маршрутов для CSS-файлов и загруженных пользователем изображений;
6. Настройка WSGI-файла для корректного запуска Flask-приложения.

После успешного деплоя приложение стало доступно по адресу:

<https://kkstyaa.pythonanywhere.com>

4.3 Оформление отчёта

Отчёт о курсовой работе оформлен в системе вёрстки Latex с использованием стилевого пакета `mospolytech_coursework`, предоставленного университетом. Структура отчёта включает следующие разделы:

- введение с обоснованием актуальности и постановкой задачи;
- анализ предметной области и обзор существующих решений;
- проектирование веб-приложения (анализ аудитории, функциональность, модель данных, макеты страниц);
- разработка веб-приложения с приведением листингов ключевых модулей;
- оформление итогов работы (Git-репозиторий, деплой на хостинг).

Все графические материалы (ER-диаграмма, Use Case-диаграмма, макеты страниц) выполнены с использованием инструментов **Draw.io** и **Mockflow**. Текст отчёта оформлен в соответствии с требованиями ГОСТ и методическими рекомендациями университета.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсового проекта было разработано веб-приложение «Книга снов» — инструмент для ведения дневника сновидений с возможностью статистического анализа записей.

В ходе работы были решены все поставленные задачи:

1. Проведён сравнительный анализ существующих решений для ведения дневников сновидений, выявлены их недостатки и обоснован выбор технологического стека (Flask, SQLite, Bootstrap 5).
2. Разработана архитектура клиент-серверного приложения, спроектирована структура базы данных, состоящая из 10 таблиц, обеспечивающих хранение пользователей, снов, символов, статистики и данных об ИИ-генерации.
3. Реализована система аутентификации и авторизации с использованием Flask-Login. Внедрена ролевая модель доступа (пользователь / администратор), администратор имеет доступ к панели управления пользователями и символами.
4. Реализован полноценный CRUD-интерфейс для управления записями снов: создание, просмотр (с пагинацией), редактирование и удаление. Каждая операция защищена проверкой прав доступа.
5. Разработан модуль агрегации данных для статистического анализа. Пользователю доступны график эмоционального фона, облако тегов, топ-5 символов, общая статистика, а также экспорт данных в CSV (как сводной статистики, так и полной книги снов).
6. Выполнено оформление итогов работы: код размещён в публичном репозитории GitHub, приложение развёрнуто на хостинге PythonAnywhere.

Приложение реализует заявленный функционал: пользователь может регистрироваться, создавать сны, добавлять к ним иллюстрации, просматривать статистику и экспортировать данные. Администратор управляет справочником символов и пользователями.

Таким образом, цель курсового проекта — проектирование и реализация клиент-серверного веб-приложения для ведения дневника свиданий — достигнута полностью.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Курс СДО. Разработка веб-приложений [Электронный ресурс]. — Режим доступа: <https://lms.mospolytech.ru/course/view.php?id=16219> (дата обращения: 19.05.2026).
2. Материалы по Latex [Электронный ресурс]. — Режим доступа: https://lms.mospolytech.ru/pluginfile.php/1353734/mod_resource/content/1/latex3days.pdf (дата обращения: 02.04.2026).
3. Курс СДО. Разработка веб-приложений [Электронный ресурс]. — Режим доступа: <https://lms.mospolytech.ru/course/view.php?id=16219> (дата обращения: 19.05.2026).
4. Dreamouy [Электронный ресурс]. — Режим доступа: <https://apps.apple.com/ru/app/dreamouy/id6751917234> (дата обращения: 05.04.2026).
5. Awoken [Электронный ресурс]. — Режим доступа: https://www.rustore.ru/catalog/app/com.lucid_dreaming.awoken (дата обращения: 05.04.2026).
6. Lucid Dreamer [Электронный ресурс]. — Режим доступа: <https://www.luciddreamer.com/> (дата обращения: 05.04.2026).
7. PythonAnywhere Documentation [Электронный ресурс]. — Режим доступа: <https://help.pythonanywhere.com/> (дата обращения: 10.06.2026).
8. Инструкция по деплою приложений на PythonAnywhere [Электронный ресурс]. — Режим доступа: <https://youtu.be/da9lYlksvOk> (дата обращения: 10.06.2026).
9. Как создать портрет пользователя [Электронный ресурс]. — Режим доступа: <https://www.uplab.ru/blog/metod-person-kak-sozdat-portret-polzovatelya/> (дата обращения: 30.04.2026).
10. User Story [Электронный ресурс]. — Режим доступа: <https://practicum.yandex.ru/blog/chto-takoe-user-story-i-kak-napisat/> (дата обращения: 06.06.2026).
11. SQLAlchemy Documentation [Электронный ресурс]. — Режим доступа: <https://www.sqlalchemy.org/> (дата обращения: 05.06.2026).

12. Bootstrap Documentation [Электронный ресурс]. — Режим доступа: <https://getbootstrap.com/> (дата обращения: 05.06.2026).
13. Chart.js Documentation [Электронный ресурс]. — Режим доступа: <https://www.chartjs.org/> (дата обращения: 05.06.2026).